CHAPITRE 4 — MIGRATIONS & BASE DE DONNÉES (VERSION FINALE COMPLÈTE)

Par Robert DIASSÉ

1. Introduction

Laravel utilise un système puissant pour gérer la base de données : **les migrations**.
Elles permettent de construire les tables sans écrire de SQL et de suivre l'évolution de la structure du projet.

Dans un projet professionnel, la base de données est un élément central.
Il est donc essentiel de la configurer correctement *avant même d'exécuter une migration*.

Ce chapitre explique :

- comment configurer MySQL (obligatoire pour le projet fil rouge),
- comment fonctionne le fichier `.env`,
- comment activer les extensions PHP nécessaires,
- comment utiliser les migrations Laravel,
- comment créer les tables de notre projet fil rouge,
- comment vérifier que tout fonctionne.

---

2. Pourquoi changer SQLite vers MySQL ?

Laravel utilise **SQLite par défaut** car c'est rapide pour démarrer.
Pour notre projet fil rouge (réservations médicales), nous devons utiliser **MySQL**, car :

- il gère mieux les relations entre tables,
- il est plus stable pour les projets réels,
- il permet d'utiliser des clés étrangères,
- il sera identique à ce que l'on utilise dans un environnement professionnel.

Donc avant de lancer les migrations, il faut **configurer MySQL correctement**.

---

3. Préparer la base de données dans MySQL

3.1 Ouvrir phpMyAdmin

Aller à l'adresse :

```
http://localhost/phpmyadmin
```

3.2 Créer la base

Cliquer sur **Nouvelle base de données**, donner un nom :

```
reservation_app
```

Choisir l'interclassement :

```
utf8mb4_general_ci
```

Cliquer sur **Créer**.

---

4. Configurer Laravel pour utiliser MySQL

Laravel lit ses informations de base de données dans le fichier :

```
mon_projet/.env
```

Ce fichier contient les paramètres de connexion.

Dans la configuration d'origine, Laravel utilise SQLite :

```
DB_CONNECTION=sqlite
...
```

Il faut désactiver SQLite et activer MySQL.

---

5. Modifier le fichier `.env`

Remplacer toute la section BD par :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
```

```
DB_PORT=3306
DB_DATABASE=reservation_app
DB_USERNAME=root
DB_PASSWORD=
```

Explication :

- `DB_CONNECTION` : type de base → ici MySQL
- `DB_HOST` : adresse du serveur → local
- `DB_PORT` : port MySQL
- `DB_DATABASE` : nom de la base créée
- `DB_USERNAME` : utilisateur MySQL de XAMPP
- `DB_PASSWORD` : vide par défaut

---

6. Activer les extensions PHP nécessaires

Sans l'extension `pdo_mysql`, Laravel **ne pourra pas** se connecter à MySQL.

6.1 Ouvrir le fichier php.ini

Chemin :

```
C:\xampp\php\php.ini
```

6.2 Chercher ces lignes et enlever le ";"

```
;extension=pdo_mysql
;extension=mysqli
```

Deviennent :

```
extension=pdo_mysql
extension=mysqli
```

Enregistrer.

6.3 Redémarrer Apache et MySQL dans XAMPP

---

7. Vider le cache de Laravel après modification du `.env`

Laravel utilise un cache pour accélérer le chargement.
Donc après modification du `.env` :

```
php artisan config:clear
php artisan cache:clear
```

---

8. Tester la connexion MySQL avant migration (important)

Ouvrir Tinker :

```
php artisan tinker
```

Puis exécuter :

```
DB::connection()->getPdo();
```

- Si aucune erreur → connexion OK
- Si erreur → problème dans `.env`, php.ini ou MySQL

---

9. Les migrations : qu'est-ce que c'est ?

Une migration est un fichier PHP qui permet :

- de créer une table,
- d'ajouter/modifier des colonnes,
- de supprimer une table,
- de gérer les relations entre tables.

Toutes les migrations se trouvent dans :

```
database/migrations/
```

Elles agissent comme un "historique" de la structure de la base.

---

10. Commandes Artisan pour les migrations

Créer une migration :

```
php artisan make:migration nom_migration
```

Exécuter toutes les migrations :

```
php artisan migrate
```

Annuler la dernière migration :

```
php artisan migrate:rollback
```

Recréer toute la base :

```
php artisan migrate:fresh
```

---

11. Tables nécessaires pour le projet fil rouge

Le système de réservation de services médicaux nécessite trois tables principales :

1. **users** (déjà créée par Laravel Breeze)
2. **services**
3. **reservations**

Nous allons créer les deux dernières.

---

12. Ajouter la colonne “role” dans la table users

Créer une migration :

```
php artisan make:migration add_role_to_users_table
```

Contenu :

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
```

```
        $table->enum('role', ['admin', 'medecin', 'patient'])->default('patient');
    });
}
```

---

13. Migration : Table “services”

Créer la migration :

```
php artisan make:migration create_services_table
```

Contenu :

```
public function up(): void
{
    Schema::create('services', function (Blueprint $table) {
        $table->id();
        $table->string('titre');
        $table->text('description')->nullable();
        $table->decimal('prix', 8, 2)->default(0);
        $table->integer('duree');
        $table->enum('statut', ['actif', 'inactif'])->default('actif');
        $table->unsignedBigInteger('medecin_id')->nullable();
        $table->timestamps();

        $table->foreign('medecin_id')->references('id')->on('users');
    });
}
```

---

14. Migration : Table “reservations”

Créer la migration :

```
php artisan make:migration create_reservations_table
```

Contenu :

```
public function up(): void
{
    Schema::create('reservations', function (Blueprint $table) {
        $table->id();
```

```
        $table->unsignedBigInteger('user_id');
        $table->unsignedBigInteger('service_id');

        $table->date('date_reservation');
        $table->time('heure_reservation');

        $table->enum('statut', [
            'en_attente',
            'confirmee',
            'annulee',
            'effectuee'
        ])->default('en_attente');

        $table->text('commentaire')->nullable();

        $table->timestamps();

        $table->foreign('user_id')->references('id')->on('users');
        $table->foreign('service_id')->references('id')->on('services');
    });
}
```

---

15. Exécuter les migrations dans MySQL

Une fois tout configuré :

```
php artisan migrate
```

Résultat attendu dans phpMyAdmin :

- users
- services
- reservations
- migrations
- password_reset_tokens (si Breeze)

---

16. Vérifier le fonctionnement

Dans phpMyAdmin :

- vérifier que les tables existent,
- vérifier les colonnes,
- vérifier les relations,
- vérifier les clés étrangères.

Si une erreur apparaît sur SQLite :
→ le `.env` n'est pas correctement modifié.

---

17. Ajouter des données par défaut (Seeder)

Créer un fichier de seed :

```
php artisan make:seeder DemoSeeder
```

Exemple de données :

```
public function run(): void
{
    User::create([
        'name' => 'Admin',
        'email' => 'admin@test.com',
        'role' => 'admin',
        'password' => bcrypt('admin123')
    ]);

    $medecin = User::create([
        'name' => 'Dr. Alpha',
        'email' => 'medecin@test.com',
        'role' => 'medecin',
        'password' => bcrypt('password')
    ]);

    Service::create([
        'titre' => 'Consultation générale',
        'description' => 'Consultation de base.',
        'prix' => 15000,
        'duree' => 30,
        'statut' => 'actif',
        'medecin_id' => $medecin->id
    ]);
}
```

Exécuter :

```
php artisan db:seed --class=DemoSeeder
```

---

18. Conclusion

À la fin de ce chapitre, l’application possède :

- un environnement MySQL configuré correctement,
- toutes les tables nécessaires au projet fil rouge,
- des relations fonctionnelles entre users, services et reservations,
- un jeu de données de départ,
- un socle prêt pour la construction des modèles et contrôleurs.